Niagara REST API — User Guide & Test Samples

Module: niagaraRestApi v1.0.1 (Niagara 4.14) · **Station:** a Niagara 4.14 test station · **Vendor:** Shanghai Gline Net Co.

Table of Contents

1. Overview & Architecture
2. Authentication
3. REST API Endpoints
   3.1 GET /api/status
   3.2 GET /api/modules
   3.3 GET /api/types
   3.4 GET /api/config
   3.5 POST /api/batch (10 Operations)
4. Verified Test Samples
   4.1 Modbus TCP Device
   4.2 MQTT + BooleanWritable
   4.3 Add + Link + Rename
   4.4 Deep Copy + Rename
   4.5 Add + Link + Unlink + Remove
5. Python Client Tools
6. Building & Deploying
7. Known Limitations
8. Alarms & History — Extensions via REST
9. Reading Alarms & History Data — Web Box

1. Overview & Architecture

The **niagaraRestApi** module is a custom Niagara N4 module that embeds an HTTP server inside a Niagara Station. It exposes a JSON-based REST API for programmatic interaction with the station — all without requiring SMA (Software Maintenance Agreement) or OEM licenses.

Architecture

```
RestApiService  (extends BComponent)
 ├─ enabled  (boolean)
 ├─ port     (int, default 8082)
 ├─ started() → Creates HttpServer
 └─ stopped() → Stops  HttpServer

Handlers:
  Status  → GET  /api/status
  Modules → GET  /api/modules
  Types   → GET  /api/types?module=xxx
  Config  → GET  /api/config?path=/xxx
  Batch   → POST /api/batch
```

- Uses the JDK built-in HttpServer — no external web container.
- JSON-only request/response format.
- **Every request must be authenticated** (HTTP Basic Auth) — see Section 2.
- Batch mutation model: all writes go through `POST /api/batch`.

2. Authentication

Basic Auth on every request

The REST API is **authenticated by default**: every request must carry HTTP Basic credentials, and a missing or wrong credential is rejected with **HTTP 401**.

```
curl -u "<USER>:<PASS>" -X POST http://<HOST>:8082/api/batch \
  -H "Content-Type: application/json" -d '[{"OP":"list","PATH":"/"}]'
```

The **only** endpoint exempt from authentication is `GET /api/status`, available as a liveness probe for monitoring:

```
# Unauthenticated liveness probe — the sole auth-exempt endpoint
curl http://<HOST>:8082/api/status
{"status":"ok","name":"glineRestApi","version":"2.4","port":8082}
```

Credentials are configurable at runtime and persist across station restarts. A **CIDR allow-list** can further restrict access to trusted network segments; a request from outside the list is rejected with **HTTP 403** (loopback is always allowed as a recovery path).

Request	Result	Note
No credentials	**401**	Rejected
Wrong credentials	**401**	Rejected
Valid credentials, outside allow-list	**403**	Rejected
Valid credentials, inside allow-list	200	Accepted

`GET /api/status` without credentials	200	Liveness probe — the only exemption

■ **Even though the API is authenticated, still do not expose it directly to the public internet — put it behind a firewall or reverse proxy.**

■ **Even with authentication enabled, do not expose the port directly to the public internet without a firewall or reverse proxy.**

3. REST API Endpoints Reference

3.1 GET /api/status

Check if the REST API service is running.

```
curl -u "<USER>:<PASS>" http://<HOST>:8082/api/status
{"status":"ok","name":"glineRestApi","version":"2.4","port":8082}
```

The port is configurable on the service component; `8082` is the default. Replace `<HOST>:8082` with your station's actual host and port.

3.2 GET /api/modules

Informational stub. This endpoint does not enumerate modules — it returns a fixed hint pointing at `/api/config`. Use `GET /api/config?path=/` to browse the station tree instead.

```
curl -u "<USER>:<PASS>" http://<HOST>:8082/api/modules
{"note":"use /api/config to browse station tree"}
```

3.3 GET /api/types

Informational stub. This endpoint returns a fixed hint listing common types; it does not enumerate the type dictionary for a module. To enumerate types, browse the station tree via `/api/config`, or run a BQL query (see Section 3.5).

```
curl -u "<USER>:<PASS>" "http://<HOST>:8082/api/types?module=control"
{"note":"use /api/config to browse types; common types: baja:Folder, ..."}
```

Common Module Types

Module	Common Types	Description
baja	Folder	Container component
control	NumericWritable, BooleanWritable, EnumWritable	Point types
modbusTcp	ModbusTcpNetwork, ModbusTcpDevice	TCP network/device
modbusCore	NumericProxyExt, BooleanProxyExt	Modbus proxy extension
kitControl	Add, Ramp, SineWave, Random	Simulation signals

3.4 GET /api/config

Browse the station configuration tree at any path.

```
curl -u "<USER>:<PASS>" "http://<HOST>:8082/api/config?path=/"
{"name":"Station","type":"Station","children":[
  {"name":"AlarmService","type":"AlarmService"},
  {"name":"Drivers","type":"Driver"}, ...]}
```

3.5 POST /api/batch — Core Operation

All mutations go through this endpoint. Send a JSON array of operation objects executed sequentially in order.

Common Fields

Field	Type	Description	Applies To
OP	string	Operation type	ALL
PATH	string	Component path from /	ALL
T	string	Type: module:TypeName	add
N	string	Component/slot name	add, mod, link
NEW	string	New name	rename, cp
V	any	Value (strings need quotes)	mod

TARGET	string	Target path	link, unlink
TARGET_N	string	Target slot	link
VAL	any	Invoke value	invoke
VAL_TYPE	string	num / bool / str	invoke
TARGET_PATH	string	Copy destination	cp

10 Operations

1. ADD — Create. Creates a child component under the parent path.

```
{"OP":"add","PATH":"/","T":"baja:Folder","N":"myFolder"}
```

2. LINK — Connect. Connects source.out to target.in10 via BConversionLink with auto-type-conversion.

```
{"OP":"link","PATH":"/src","N":"out","TARGET":"/tgt","TARGET_N":"in10"}
```

3. RENAME — Rename. Renames using parent.rename() — preserves all existing links (handle-based ORD).

```
{"OP":"rename","PATH":"/folder/old","NEW":"new"}
```

4. CP — Deep Copy. Recursive deep copy of entire subtree. Internal links rebuilt in copy.

```
{"OP":"cp","PATH":"/folder","NEW":"copy","TARGET_PATH":"/"}
```

5. MOD — Modify Value. Writes a property value. Strings need quotes; numbers bare; booleans true/false.

```
{"OP":"mod","PATH":"/device","N":"ipAddress","V":"<IP>"}
```

6. UNLINK — Disconnect. Removes the Link child from the target point.

```
{"OP":"unlink","TARGET":"/tgtPoint"}
```

7. REMOVE — Delete. Permanently removes the component from the station tree.

```
{"OP":"remove","PATH":"/parent/child"}
```

8. READ — Read Property. Returns the current value of the named slot. Omit `N` to get the component's **name and type** (to enumerate slots and children, use `/api/config?path=…` or the `LIST` operation).

```
{"OP":"read","PATH":"/device","N":"ipAddress"}
```

9. LIST — List Children. Returns names and types of all immediate child components.

```
{"OP":"list","PATH":"/myFolder"}
```

10. INVOKE — Call Action. Invokes a Niagara action. VAL_TYPE supports: num, bool, str.

```
{"OP":"invoke","PATH":"/point/set","VAL":"42","VAL_TYPE":"num"}
```

MOD Special Values

Type	JSON Format	Example
String	`V:"value"`	`V:"<IP>"`
Integer	`V:number`	`V:502`
Boolean	`V:true/false`	`V:true`
FlexAddress	`V:"addr|format"`	`V:"2|decimal"`

- Enum types (regType, dataType) cannot be written via MOD (BString-to-enum conversion fails).
- BFacets types (deviceFacets) cannot be written via MOD.

4. Verified Test Samples

All operations were tested on a Niagara 4.14 station with default admin credentials.

4.1 Create Modbus TCP Device (Thermostat)

Creates a Modbus TCP device with 3 points (OnOff, Mode, SetTemp), configures register addresses, sets a device IP address, and pings to verify connectivity.

```
curl -u "<USER>:<PASS>" -X POST http://<HOST>:8082/api/batch -H "Content-Type: application/json" -d '[
{"OP":"add","PATH":"/Drivers/ModbusTcpNetwork","T":"modbusTcp:ModbusTcpDevice","N":"T8600_new"},
{"OP":"mod","PATH":"/.../T8600_new","N":"deviceAddress","V":2},
{"OP":"mod","PATH":"/.../T8600_new","N":"ipAddress","V":"<IP>"},
{"OP":"add","PATH":"/.../T8600_new/points","T":"control:NumericWritable","N":"OnOff"},
{"OP":"add","PATH":"/.../OnOff","T":"modbusCore:ModbusClientNumericProxyExt","N":"proxyExt"},
{"OP":"mod","PATH":"/.../OnOff/proxyExt","N":"dataAddress","V":"2|decimal"},
{"OP":"add","PATH":"/.../T8600_new/points","T":"control:NumericWritable","N":"Mode"},
{"OP":"add","PATH":"/.../Mode","T":"modbusCore:ModbusClientNumericProxyExt","N":"proxyExt"},
{"OP":"mod","PATH":"/.../Mode/proxyExt","N":"dataAddress","V":"3|decimal"},
{"OP":"add","PATH":"/.../T8600_new/points","T":"control:NumericWritable","N":"SetTemp"},
{"OP":"add","PATH":"/.../SetTemp","T":"modbusCore:ModbusClientNumericProxyExt","N":"proxyExt"},
{"OP":"mod","PATH":"/.../SetTemp/proxyExt","N":"dataAddress","V":"4|decimal"},
{"OP":"invoke","PATH":"/Drivers/ModbusTcpNetwork/T8600_new/ping"}]'
```

4.2 MQTT + BooleanWritable Points

Creates 10 BooleanWritable points under the MQTT folder, then links each to the MqttClient output.

```
[{"OP":"add","PATH":"/MQTT","T":"control:BooleanWritable","N":"Point1"}, ... ,
 {"OP":"add","PATH":"/MQTT","T":"control:BooleanWritable","N":"Point10"}]
```

4.3 Add + Link + Rename Flow

Test: create a folder with two NumericWritable points, link them, then rename the source. Verify the link survives rename (because ORD uses handle `h:xxxxx`, not name).

```
[{"OP":"add","PATH":"/","T":"baja:Folder","N":"testFolder"},
 {"OP":"add","PATH":"/testFolder","T":"control:NumericWritable","N":"src"}, ...]
```

Result: renamedSrc still linked to tgtPoint.in10

4.4 Deep Copy (CP) + Rename

Create a folder with internal links, deep copy to a new location. Verify internal links are rebuilt in the copy.

```
[{"OP":"add","PATH":"/","T":"baja:Folder","N":"original"},
 {"OP":"add","PATH":"/original","T":"control:NumericWritable","N":"point1"}, ...]
```

Result: copy has copy.point1 link to copy.point2

4.5 Add + Link + Unlink + Remove (Full Lifecycle)

Complete component lifecycle: create, link, unlink (disconnect), and delete.

```
[{"OP":"add","PATH":"/","T":"baja:Folder","N":"lifecycle"},
 {"OP":"add","PATH":"/lifecycle","T":"control:NumericWritable","N":"src"}, ...]
```

Result: unlink removes Link child; src deleted; dst remains orphaned

5. Python Client Tools

5.1 niagara-rest-client.py

Universal REST client supporting: login, status, config browse, BQL query, history extraction, point override, and alarm export.

```
python niagara-rest-client.py <host:port> --user <USER> --password <PASS> --status
python niagara-rest-client.py <host:port> --user <USER> --password <PASS> --batch batch.json
```

5.2 niagara-bql-query.py

BQL CLI tool with CSV export, type filtering, and configurable parent path.

```
python niagara-bql-query.py --host <HOST> --user <USER> --password <PASS> \
  --query "SELECT * FROM control:BooleanWritable"
```

5.3 alarm_to_csv.py

Dual-approach alarm export: ProgramService BSH script execution or external Python HTTP client.

```
python alarm_to_csv.py --host <HOST> --user <USER> --password <PASS> --output alarms.csv
```

5.4 Station Exploration Scripts

- **explore_station.py:** REST API-based station exploration (status, config browse, services).
- **explore_station_v2.py:** Full SCRAM auth + integrated XML parsing with tree search into Modbus, BACnet, and simulation folders.
- **test_connection.py:** Quick connectivity test against a target host.

6. Building & Deploying

1. Build the module JAR from source against the Niagara 4.14 SDK.
2. Copy the JAR into the station host's `modules/` directory.
3. Restart the station (or the Niagara service) so the module loads.
4. The REST API is then live on the configured port (default **8082** for v2).

> Test the module on a staging station before applying it to production.

7. Known Limitations

1. **Authentication required** — every request needs valid Basic Auth credentials; only `GET /api/status` is exempt. Do not expose the port directly to the public internet.
2. **Enum Properties** (regType, dataType) — cannot write via MOD (BString-to-enum conversion fails).
3. **BFacets Properties** (deviceFacets) — cannot write via MOD.
4. **Module Reload** — disabling RestApiService requires a full station restart for the module to reload.
5. **No Alarm/History Endpoints** — the REST API has no dedicated alarm or history **query** endpoints. Configure extensions via REST (Section 8) and read the data via **Web Box** (Section 9).

8. Alarms & History — Extensions via REST

Alarm and history capture is configured by attaching **extensions** to a point. These are ordinary `BComponent` children, so they can be created and configured through the REST API — no GUI required.

Extension types

Type	Purpose
`alarm:AlarmSourceExt`	Alarm source extension for a point
`history:NumericCovHistoryExt`	Numeric history, change-of-value
`history:BooleanCovHistoryExt`	Boolean history, change-of-value
`history:EnumCovHistoryExt`	Enum history, change-of-value
`history:StringCovHistoryExt`	String history, change-of-value
`history:NumericIntervalHistoryExt`	Numeric history, fixed interval

Attach a history extension and enable it

```
curl -u "<USER>:<PASS>" -X POST http://<HOST>:8082/api/batch \
  -H "Content-Type: application/json" -d '[
{"OP":"add","PATH":"/myFolder/pt","T":"history:NumericCovHistoryExt","N":"he"},
{"OP":"mod","PATH":"/myFolder/pt/he","N":"enabled","V":true}
]'
```

After `enabled=true` the extension reports `active: true` — history collection is live.

Attach an alarm extension and configure it

```
curl -u "<USER>:<PASS>" -X POST http://<HOST>:8082/api/batch \
  -H "Content-Type: application/json" -d '[
{"OP":"add","PATH":"/myFolder/pt","T":"alarm:AlarmSourceExt","N":"ae"},
{"OP":"mod","PATH":"/myFolder/pt/ae","N":"alarmClass","V":"/AlarmClass"},
{"OP":"mod","PATH":"/myFolder/pt/ae","N":"timeDelay","V":5000}
]'
```

Known MOD limits for extensions

Slot	Behaviour
`enabled`	Boolean — writable; activating a history extension
`alarmClass`	String — writable
`timeDelay`	`RelTime` — write a **raw millisecond number**, not `"5s"`
`historyName` / `toOffnormalText`	`BFormat` — **cannot** be written via MOD
`alarmEnable`	`AlarmTransitionBits` — **cannot** be written via MOD

Extensions created via REST remain fully visible and editable in Workbench.

9. Reading Alarms & History Data — Web Box

The REST API deliberately has no alarm or history endpoints. That data is available through the station's **Web Box** feature, which can be driven programmatically for automated extraction.

Web Box query endpoints

Endpoint	Method	Purpose
`/wsbox/query/data/<ord>`	GET	Read history records for a point (paged, sortable)
`/wsbox/query/source/<ord>`	GET	History point source / metadata
`/wsbox/query/lastTime/<ord>`	GET	Timestamp of the most recent record

Query parameters

- `offset` / `limit` — pagination window.
- `sortColumn=timestamp` and `descending=false` — chronological order.
- `timeRange=startTime=…;endTime=…` — bounded time window.

Slot path encoding

History points are addressed by an **ord** of the form `history:/<station>/<historyName>`. When passing an ord in a URL it must be escaped — the mapping includes `:` → `$3a`, `/` → `$2f`, `?` → `$3f`, `=` → `$3d`, `&` → `$26`, `$` → `$24`.

> ■ **Escaping `&` is mandatory.** A raw `&` is parsed as an HTTP parameter separator and the query silently returns an empty result set.

Automation notes

- Requests must be sent as **GET**.
- The response is a chunked **NDJSON** stream — read it with a streaming client (e.g. browser `XMLHttpRequest` with `onprogress`), not a single blocking read.
- The CSRF token must be obtained inside the station page context (`csrfUtil.getCsrfToken()`) and sent in the `x-niagara-csrfToken` header; the station session must already be authenticated.

With these pieces in place, alarm and history extraction can be fully automated — no fixed REST endpoint is required for it.

For commercial customization, integration, or deployment support, please contact:

Shanghai Gline Net Co.
Email: jason.zhang@gline-net.com
Tel / WhatsApp: +86 138 0199 0968

© 2026 Shanghai Gline Net Co. — End of Document